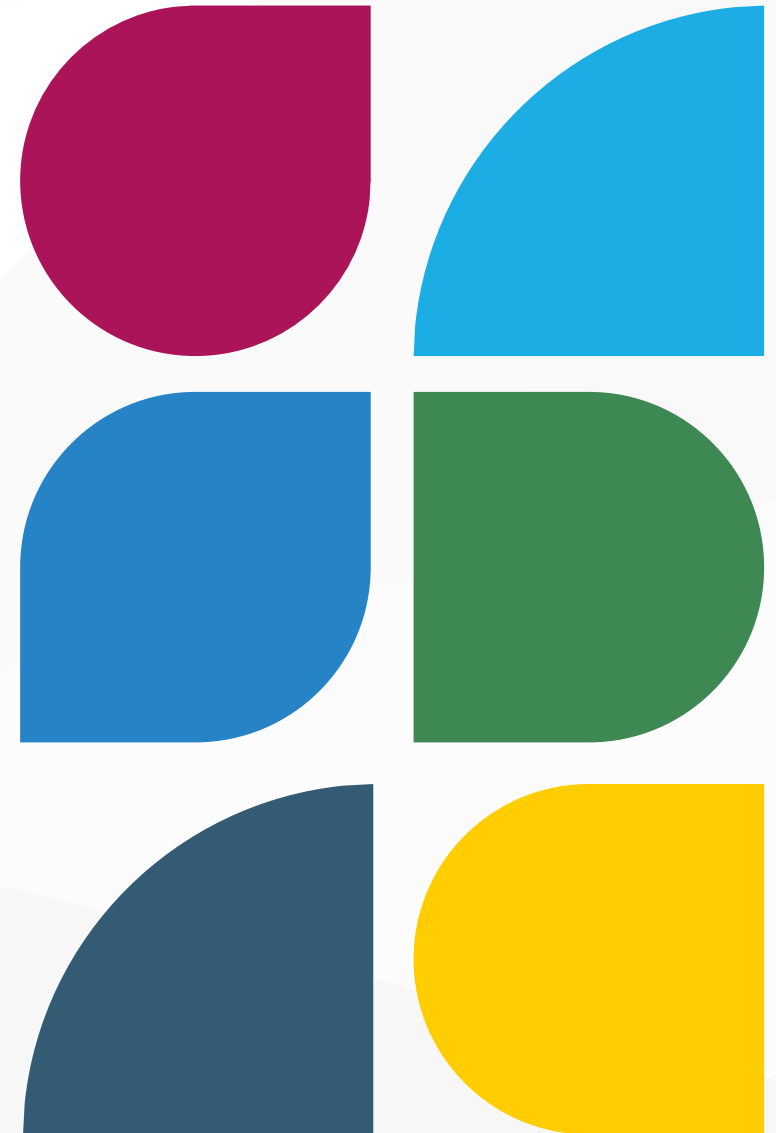
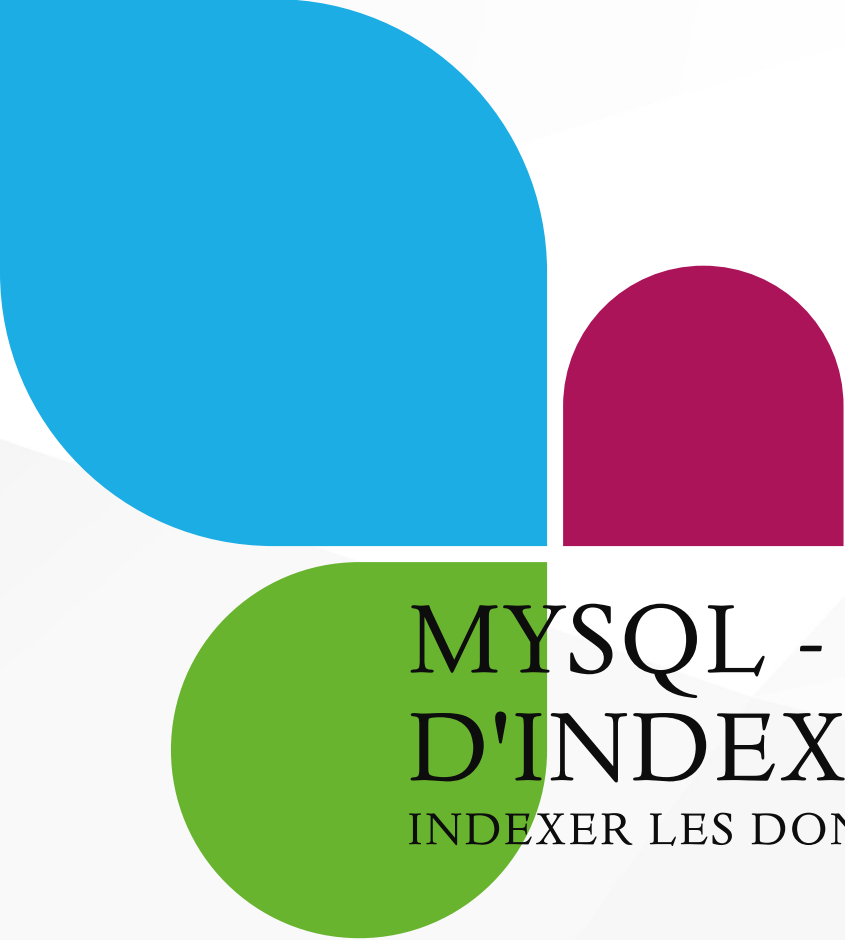


LE LANGAGE SQL STRUCTURED QUERY LANGUAGE

Fonctionnalités complémentaires de SQL : Index,
transaction, fonctions et procédures.





MYSQL - CRÉATION D'INDEX

INDEXER LES DONNÉES

DÉFINITION

Un index est une structure qui reprend la liste ordonnée des valeurs auxquelles il se rapporte.

Les index sont utilisés pour :

- accélérer les requêtes (notamment les requêtes impliquant plusieurs tables, ou les requêtes de recherche).
- permettre de garantir l'intégrité des données de la base. [indispensables à la création de clés, étrangères et primaires]
- accélérer les requêtes qui utilisent des colonnes indexées comme critères de recherche.

Convention de nommage

Il n'existe pas de convention de nommage spécifique sur le nom des index, juste des suggestions de quelques développeurs et administrateurs de bases de données.

Voici une liste de suggestions de préfixes à utiliser pour nommer un index :

Préfixe "PK_" pour Primary Key (traduction : clé primaire)

Préfixe "FK_" pour Foreign Key (traduction : clé étrangère)

Préfixe "UK_" pour Unique Key (traduction : clé unique)

Préfixe "UX_" pour Unique Index (traduction : index unique)

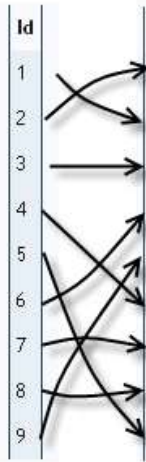
Préfixe "IX_" pour chaque autre Index

FONCTIONNEMENT

Les index **ne sont pas des objets** qui font partie de la théorie relationnelle. Ils **sont des objets physiques** qui permettent d'accélérer l'accès aux données.

- Et comme ils ne sont que des moyens d'optimisation des accès, les index ne font pas non plus partie de la norme SQL.
- C'est d'ailleurs pour cette raison que la syntaxe de création d'index est si différente d'une base de données à une autre.

L'index permet de pointer l'endroit de la table où se trouve une donnée, pour y accéder directement. Parfois c'est toute une plage de l'index, voire sa totalité, qui sera lue, ce qui est généralement plus rapide que lire toute la table.



	Id	Espèce	Sexe	Date de naissance	Nom	Commentaires
1	2	chat	NULL	2010-03-24 02:23:00	Roucky	NULL
2	1	chien	male	2010-04-05 13:43:00	Rox	Mordille beaucoup
3	3	chat	femelle	2010-09-13 15:02:00	Schtroumpfette	NULL
4	6	tortue	femelle	2009-06-13 08:17:00	Bobosse	Carapace bizarre
5	9	tortue	NULL	2010-08-23 05:18:00	NULL	NULL
6	4	tortue	femelle	2009-08-03 05:12:00	NULL	NULL
7	7	chien	femelle	2008-12-06 05:18:00	Caroline	NULL
8	8	chat	male	2008-09-11 15:38:00	Bagherra	NULL
9	5	chat	NULL	2010-10-03 16:44:00	Choupi	Né sans oreille gauche

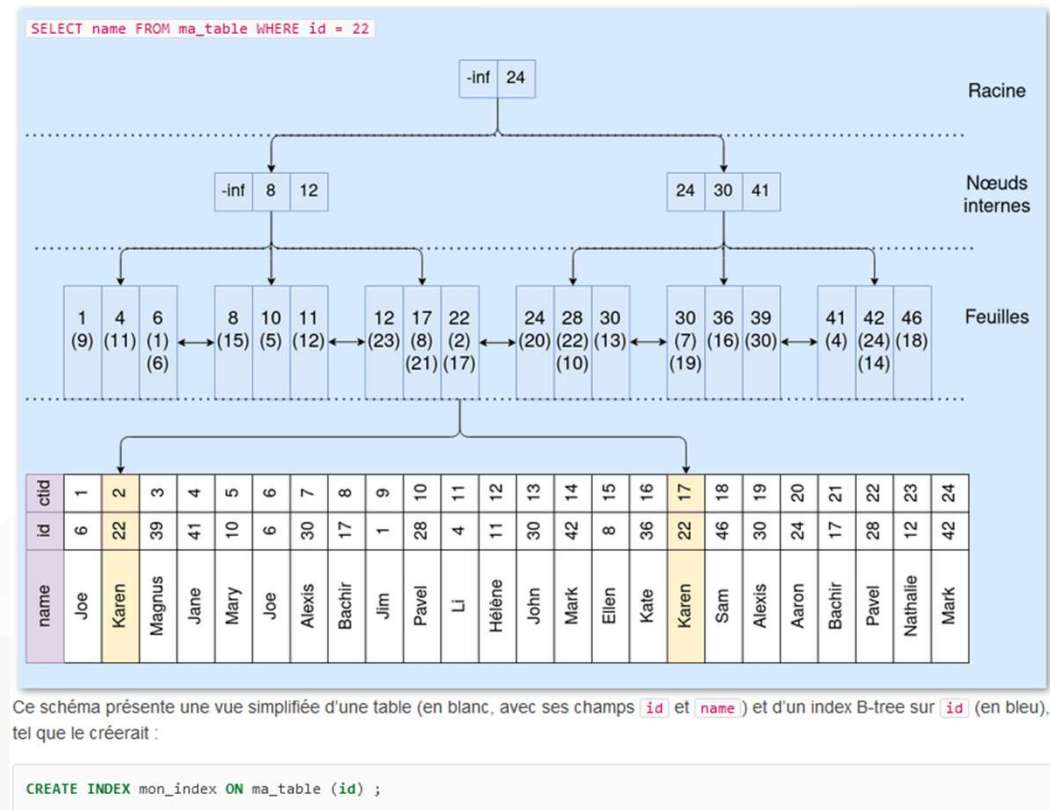
Représentation simplifiée

INDEX B-TREE

MySQL stocke les index dans une structure de type "arbre" dit **index B-tree** contenant trois types de nœuds

- **la racine** : elle est unique c'est la base de l'arbre
- **des nœuds internes** : il peut y en avoir plusieurs niveaux
- **des feuilles** : elles contiennent :
 - les **valeurs indexées** (triées !)
 - les **valeurs incluses** (si applicable)
 - les **positions physiques (ctid)**, ici entre parenthèses et sous forme abrégée, car la forme réelle est (numéro de bloc, position de la ligne dans le bloc)
 - l'adresse de la feuille précédente et de la feuille suivante.

La racine et les nœuds internes contiennent des enregistrements qui décrivent la valeur minimale de chaque bloc du niveau inférieur et leur adresse (ctid)



LES INDEX

Créer un index ordinaire

```
-- La syntaxe basique pour créer un index est la suivante :  
CREATE INDEX `index_nom` ON `table`;  
  
/*  
Créer un index sur une seule colonne en précisant la colonne à indexer  
*/  
CREATE INDEX `index_nom` ON `table` (`colonne1`);  
  
-- Pour insérer un index sur plusieurs colonnes  
CREATE INDEX `index_nom` ON `table` (`colonne1`, `colonne2`);
```

Créer un index unique

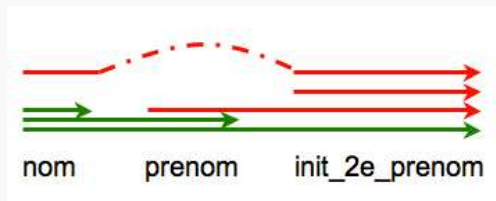
```
/*  
Un index unique permet de spécifier qu'une ou plusieurs colonnes doivent  
contenir des valeurs uniques à chaque enregistrement. Le système de base de  
données retournera une erreur si une requête tente d'insérer des données qui  
feront doublons sur la clé d'unicité.  
*/  
  
-- Pour créer ce type d'index  
CREATE UNIQUE INDEX `index_nom` ON `table` (`colonne1`);  
/*  
Dans cet exemple un index unique sera créé sur la colonne nommée colonne1. Cela  
signifie qu'il ne peut pas y avoir plusieurs fois la même valeur sur 2  
enregistrements distincts contenus dans cette table.  
*/  
  
-- Pour Créer un index d'unicité sur 2 colonnes  
CREATE UNIQUE INDEX `index_nom` ON `table` (`colonne1`, `colonne2`);
```

LES INDEX

Index sur plusieurs colonnes

Si on effectue souvent des recherches à la fois sur le nom, le prénom ? On va créer un index sur les deux colonnes.

MySQL est capable de tirer parti de votre index sur (nom, prénom) pour certaines autres recherches : "index par la gauche"



Index sur des colonnes de type alphanumérique

Types CHAR et VARCHAR : l'index décompose votre varchar caractères par caractères. On peut créer l'index à partir d'une certaine taille pour éviter de prendre la totalité de la chaîne de caractères.

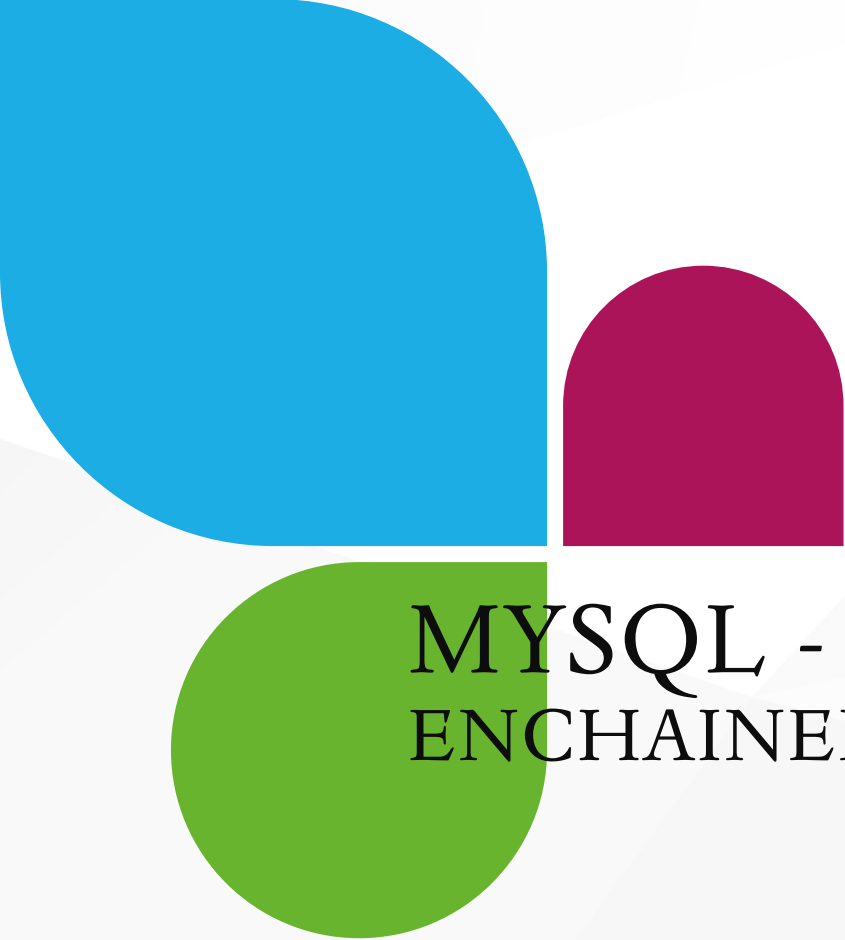
DÉSAVANTAGES

On peut noter quelques désavantages de ce type d'objet en SQL.

1. Ils prennent de la place en mémoire.
2. Ils ralentissent les requêtes d'insertion, modification et suppression, puisqu'à chaque fois, il faut remettre l'index à jour en plus de la table.

Par conséquent, n'ajoutez pas d'index lorsque ce n'est pas vraiment utile.

3. Avoir un index UNIQUE sur une colonne (ou plusieurs) permet de s'assurer que jamais vous n'insérerez deux fois la même valeur (ou combinaison de valeurs) dans la table.
4. Lorsque vous mettez un index UNIQUE sur une table, vous ne mettez pas seulement un index, vous ajoutez surtout une **contrainte**.



MYSQL - TRANSACTIONS

ENCHAINEMENT DE REQUÊTES

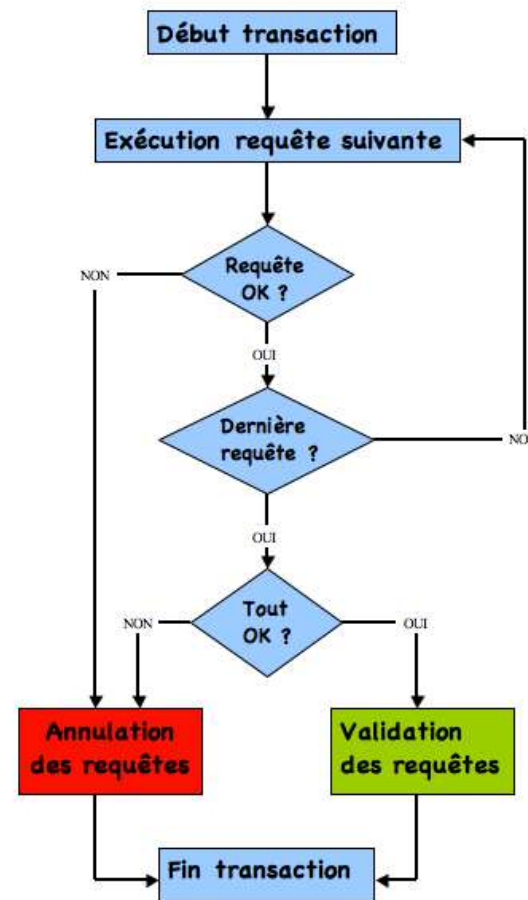
PRINCIPE

Une transaction :

- ensemble de requêtes qui sont exécutées en un seul bloc.
- si une des requêtes du bloc échoue, on peut décider d'annuler tout le bloc de requêtes (ou de quand même valider les requêtes qui ont réussi).



- *Les tables MyISAM sont non transactionnelles, donc ne supportent pas les transactions.*
- *les tables InnoDB sont transactionnelles, donc supportent les transactions.*



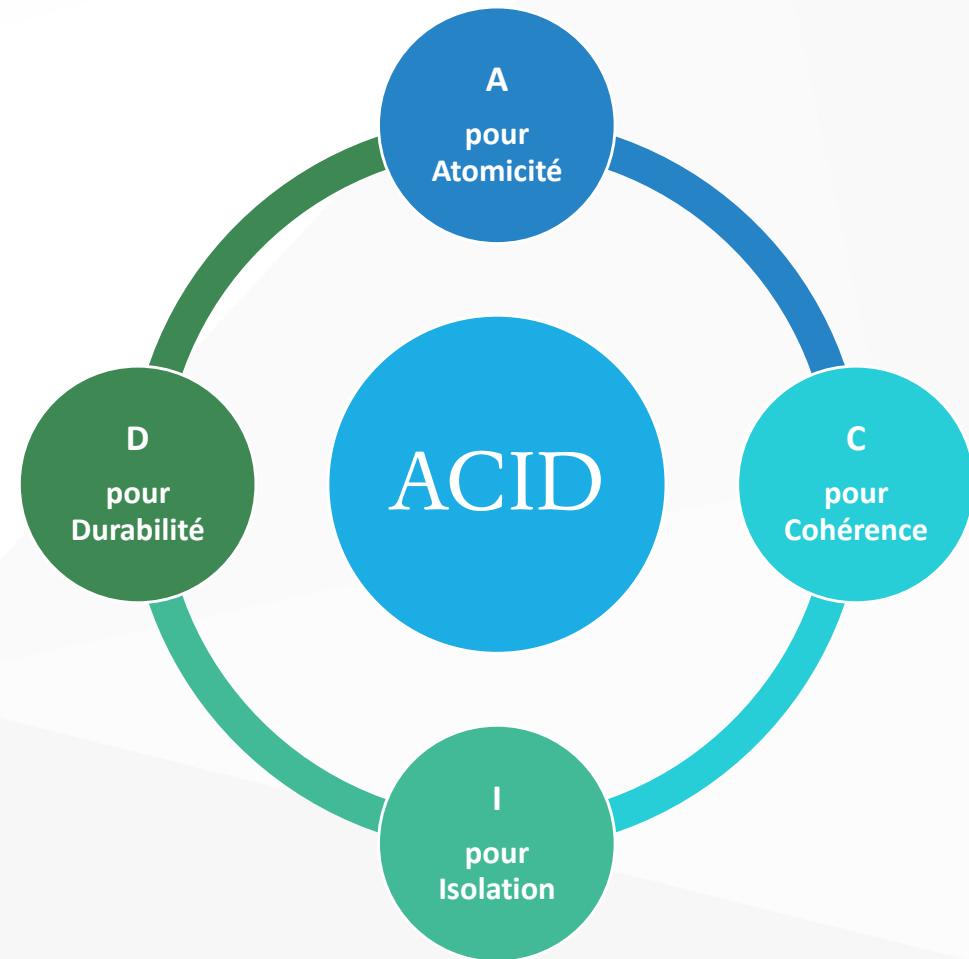
PRÉCISIONS SUR LES TRANSACTIONS

Toutes les commandes qui influent sur la structure de la base valident implicitement les transactions.

- CREATE DATABASE, DROP DATABASE ;
- CREATE TABLE, ALTER TABLE, RENAME TABLE, DROP TABLE, CREATE INDEX, DROP INDEX, CREATE PROCEDURE etc...



Les transactions doivent respecter les propriétés ACID qui garantissent qu'une transaction est exécutée de façon fiable.





PRINCIPE ACID

Atomicité

L'atomicité garantit que chaque transaction est traitée comme une seule "unité", **qui réussit complètement ou échoue complètement** :

- si l'une des déclarations constituant une transaction échoue, la transaction entière échoue et la base de données reste inchangée.

Cohérence

La cohérence garantit qu'une transaction ne peut faire passer la base de données que d'un état cohérent à un autre, en préservant les invariants de la base de données :

- toute donnée écrite dans la base de données doit être valide selon toutes les règles définies, y compris les contraintes, les rollbacks, les déclencheurs et toute combinaison de ceux-ci.

Cela empêche la corruption de la base de données par une transaction illégale.



PRINCIPE ACID

Isolation

L'isolation garantit que l'exécution simultanée des transactions laisse la base de données dans le même état que celui qui aurait été obtenu si les transactions avaient été exécutées séquentiellement.

Durabilité

La durabilité garantit qu'une fois qu'une transaction a été validée, elle le restera même en cas de défaillance du système (par exemple, une panne de courant ou un crash).

Cela signifie généralement que les transactions terminées (ou leurs effets) sont enregistrées dans une mémoire non volatile.

FONCTIONNEMENT DANS INNODB

Dans InnoDB, toutes les activités des utilisateurs se produisent à l'intérieur d'une transaction.

- Si autocommit est activé, chaque instruction SQL forme une seule transaction.
- Par défaut, MySQL démarre la session pour chaque nouvelle connexion avec autocommit activé, donc MySQL effectue un commit après chaque instruction SQL si cette instruction n'a pas renvoyé d'erreur.
- Si une instruction renvoie une erreur, le comportement de validation ou de restauration dépend de l'erreur.
- Si autocommit est désactivé dans une session avec `SET autocommit = 0`, la session a toujours une transaction ouverte. Une instruction `COMMIT` ou `ROLLBACK` termine la transaction en cours et une nouvelle commence.
- Un `COMMIT` signifie que les modifications apportées à la transaction en cours sont rendues permanentes et deviennent visibles pour les autres sessions.
- Un `ROLLBACK` instruction, en revanche, annule toutes les modifications apportées par la transaction en cours. Les deux `COMMIT` et `ROLLBACK` libèrent tous les verrous qui ont été définis pendant la transaction en cours.

SYNTAXE ET UTILISATION

START TRANSACTION ou **BEGIN** : démarrer une nouvelle transaction.

COMMIT valide la transaction en cours, rendant ses modifications permanentes.

ROLLBACK annule la transaction en cours, annulant ses modifications.

SET autocommit désactive ou active le mode de validation automatique par défaut pour la session en cours.



*Par défaut, MySQL s'exécute avec le mode de validation automatique activé. Cela signifie que, lorsqu'elle n'est pas autrement à l'intérieur d'une transaction, chaque instruction est atomique, comme si elle était entourée de **START TRANSACTION** et **COMMIT**.*

*Vous ne pouvez pas utiliser **ROLLBACK** pour annuler l'effet ; cependant, si une erreur se produit lors de l'exécution de l'instruction, l'instruction est annulée.*

```
/*
ce n'est valable que pour la session courante.
Or, en ouvrant une nouvelle connexion, vous avez créé une nouvelle session.
Il faut donc désactiver donc à nouveau ce mode.
*/
SET autocommit=0;

START TRANSACTION;

-- debut de nos requêtes
INSERT INTO...

/*
Une fois la transaction ouverte, les requêtes devront être validées pour
prendre effet.
Attention au fait qu'un COMMIT ou un ROLLBACK met fin automatiquement à la
transaction, donc les commandes suivantes seront à nouveau « commitées »
automatiquement si une nouvelle transaction n'est pas ouverte.
*/
COMMIT ou ROLLBACK;
```

LES JALONS

Il est possible de démarrer une transaction à l'intérieur d'une transaction.

On peut poser des jalons de transaction :

- points de repère qui permettent d'annuler toutes les requêtes exécutées depuis ce jalon, et non toutes les requêtes de la transaction.

```
SAVEPOINT nom_jalon;  
-- Crée un jalon avec comme nom "nom_jalon"  
  
ROLLBACK [WORK] TO [SAVEPOINT] nom_jalon;  
-- Annule les requêtes exécutées depuis le jalon "nom_jalon",  
-- WORK et SAVEPOINT ne sont pas obligatoires  
  
RELEASE SAVEPOINT nom_jalon;  
-- Retire le jalon "nom_jalon"  
-- (sans annuler, ni valider les requêtes faites depuis)
```


LES VEROUS

Complément indispensable des transactions, les verrous permettent de sécuriser les requêtes en bloquant ponctuellement et partiellement l'accès aux données.

2 types de verrous sont disponibles :

- Les verrous de tables
- Les verrous de lignes

```
-- Verrous de tables

LOCK TABLES nom_table [AS alias_table] [READ | WRITE] [, ...];

LOCK TABLES Espece READ, Adoption AS adopt WRITE;
-- On pose un verrou de lecture sur Espece
-- et un verrou d'écriture sur Adoption avec l'alias adopt

-- Verrous de lignes

SELECT * FROM Animal WHERE espece_id = 5 LOCK IN SHARE MODE;
/*
pose donc un verrou partagé sur les lignes de la table Animal pour lesquelles
espece_id vaut 5.
Ce verrou signifie en fait, pour les autres sessions : "Je suis en train de
lire ces données. Vous pouvez venir les lire aussi, mais pas les modifier tant
que je n'ai pas terminé."
*/

SELECT * FROM Animal WHERE espece_id = 5 FOR UPDATE;
/*
poser un verrou exclusif, on utilise FOR UPDATE à la fin de la requête SELECT.
Ce verrou signifie aux autres sessions : "Je suis en train de lire ces données
dans le but probable de faire une modification. Ne les lisez pas avant que
j'aie fini (et bien sûr, ne les modifiez pas)."
*/
```

VALIDATION IMPLICITE



Toutes les commandes qui créent, modifient, suppriment des objets dans la base de données valident implicitement les transactions.

Cela comprend donc :

- la création et suppression de bases de données : `CREATE DATABASE`, `DROP DATABASE` ;
- la création, modification, suppression de tables : `CREATE TABLE`, `ALTER TABLE`, `RENAME TABLE`, `DROP TABLE` ;
- la création, modification, suppression d'index : `CREATE INDEX`, `DROP INDEX` ;
- la création d'objets comme les procédures stockées, les vues, etc., dont nous parlerons plus tard.

La création, la modification et la suppression d'utilisateurs provoquent aussi une validation implicite.

la commande `START TRANSACTION` provoque également une validation implicite si elle est exécutée à l'intérieur d'une transaction. Le fait d'activer le mode autocommit (s'il n'était pas déjà activé) a le même effet.

La création et suppression de verrous de table clôturent aussi une transaction en la validant implicitement.



MYSQL - LES FONCTIONS

ÉCRIRE NOS PROPRES FONCTIONS

LES FONCTIONS

Définition

Une fonction :

- code qui effectue une série d'instructions bien précises et renvoie le résultat de ces instructions.
- définie par son nom et ses paramètres.
- peut avoir un ou plusieurs paramètres, ou n'en avoir aucun.
- L'ordre de ces paramètres est très important.

Lorsque l'on utilise une fonction, on dit que l'on fait appel à celle-ci avec **SELECT**

- il suffit donc de donner son nom, suivi des paramètres éventuels entre parenthèses (obligatoires, même s'il n'y a aucun paramètre).

Quelques exemples

```
-- Fonction sans paramètre
SELECT PI();      -- renvoie le nombre Pi, avec 5 décimales
-- Fonction avec un paramètre
SELECT MIN(prix) AS minimum
FROM Espece;
-- Fonction avec plusieurs paramètres
SELECT REPEAT('fort ! Trop ', 4);
-- répète une chaîne (ici : 'fort ! Trop ', répété 4 fois)
```

LES FONCTIONS D'AGRÉGATION

Définition

Les fonctions d'agrégation dans le langage SQL permettent d'effectuer des opérations statistiques sur un ensemble d'enregistrement.

Étant donné que ces fonctions s'appliquent à plusieurs lignes en même temps, elles permettent des opérations qui servent à récupérer l'enregistrement le plus petit, le plus grand ou bien encore de déterminer la valeur moyenne sur plusieurs enregistrements.

Fonctions d'agrégation statistiques

Les fonctions d'agrégation sont des fonctions idéales pour effectuer quelques statistiques de bases sur des tables.

Les principales fonctions sont les suivantes :

- AVG()
- COUNT()
- MAX()
- MIN()
- SUM()

LES FONCTIONS D'AGRÉGATION

Utilisation

```
SELECT fonction(colonne) FROM table;
```

La fonction COUNT() possède une subtilité - Pour compter le nombre total de ligne d'une table, il convient d'utiliser l'étoile * :

```
SELECT COUNT(*) FROM table
```

Utilisation avec GROUP BY

Toutes ces fonctions prennent toutes leur sens lorsqu'elles sont utilisée avec la commande GROUP BY qui permet de filtrer les données sur une ou plusieurs colonnes.

LES FONCTIONS SCALAIRES

Définition

Les fonctions scalaires s'appliquent à chaque ligne indépendamment.

Il existe trois types de fonction scalaire SQL :

- les fonctions de date/heure,
- les fonctions de conversion
- les fonctions de chaîne.

Fonctions DATE et HEURE

SQL offre toute une panoplie de fonctions pouvant convertir, manipuler les types dates et heures dont voici quelques exemples :

- DATEDIFF() : qui donne un résultat en nombre de jours.
- TIMEDIFF() : qui donne un résultat sous forme de TIME.
- TIMESTAMPDIFF() : qui donne le résultat dans l'unité de temps souhaitée (heure, secondes, mois...).
- DATEDIFF(date1, date2)
- TIMEDIFF()
- ADDDATE() : qui s'utilise avec un INTERVAL ou un nombre de jours.

FONCTIONS CHAÎNES DE CARACTÈRES

Les fonctions SQL sur les chaînes de caractères permettent d'ajouter de nombreuses fonctionnalités aux requêtes SQL.

Ces fonctions sont mono-lignes : cela signifie qu'elles ne s'appliquent qu'à une seule ligne en même temps.

- `CHAR_LENGTH()` permet de compter le nombre de caractères.
- `CONCAT()` concaténer plusieurs chaînes de caractères
- `FORMAT()`
- `LENGTH()` retourner la longueur d'une chaîne
- `LOWER()` transformer la chaîne pour tout retourner en minuscule.
- `LTRIM()` supprimer les caractères vides au début de la chaîne.
- `REPLACE()` remplacer des caractères par d'autres caractères.
- `SUBSTR()` retourne un segment de chaîne.
- `SUBSTRING()` retourne un segment de chaîne.
- `TRIM()` supprime les caractères vides en début et fin de chaîne.
- `UPPER()` tout retourner en majuscule.

FONCTIONS UTILISATEUR

Définition

Les fonctions définies par l'utilisateur ne permettent pas d'exécuter des actions qui modifient l'état des bases de données.

Elle cherche une donnée, calcule une donnée et la retourne.

Création et suppression d'une fonction

```
CREATE FUNCTION <foo> ([params..])  
  RETURNS <datatype>  
  [ [NOT] DETERMINISTIC ]  
  BEGIN  
    <code>  
  RETURN <code>  
  END  
  
DROP FUNCTION [if exists] <foo>
```



ATTRIBUTS DE FONCTIONS

DERTERMINISTIC, NOT DETERMINISTIC,
NO SQL ET READS SQL DATA

FONCTIONS UTILISATEUR

Fonctions déterministes

Les fonctions déterministes retournent toujours le même résultat chaque fois qu'elles sont appelées avec un même ensemble spécifique de valeurs d'entrée et donnent le même état de la base de données.

Par exemple, la fonction `AVG()`.

Côté serveur, il conserve le résultat pour l'entrée concernée et ainsi vous retourne ce résultat pour la même entrée sans devoir refaire le calcul.

- Gain de temps et de performances.

Fonctions non déterministes

Les fonctions non déterministes peuvent retourner des résultats différents chaque fois qu'ils sont appelés avec un même ensemble spécifique de valeurs d'entrée, même si l'état de base de données auquel ils accèdent reste le même.

Par exemple, la fonction `GETDATE()`.

FONCTIONS UTILISATEUR

NO SQL

Cet attribut indique que la fonction ne contient aucune instruction SQL.

Cela signifie que la fonction ne lit ni ne modifie les données dans les tables de la base de données.

```
CREATE FUNCTION pi_value() RETURNS DECIMAL(10,8)
NO SQL
RETURN 3.14159265;
```

READ SQL DATA

Cet attribut indique que la fonction peut lire des données depuis la base de données, mais qu'elle ne modifie pas ces données.

Cela inclut les instructions SQL comme SELECT, mais exclut les instructions de modification telles que UPDATE, INSERT, ou DELETE.

RÉSUMÉ DES COMBINAISONS POSSIBLES

Les attributs DETERMINISTIC, NOT DETERMINISTIC, NO SQL, et READS SQL DATA en MySQL permettent de spécifier comment une fonction stockée interagit avec les données et si elle est déterministe ou non.

1. **DETERMINISTIC + NO SQL** : La fonction ne dépend pas de la base de données, elle est prédictible et ne contient pas de requêtes SQL.
2. **DETERMINISTIC + READS SQL DATA** : La fonction lit des données mais reste prédictible (par exemple, elle lit des valeurs constantes dans une table).
3. **NOT DETERMINISTIC + NO SQL** : Rare, mais peut se produire si la fonction dépend de facteurs externes comme l'heure du système.
4. **NOT DETERMINISTIC + READS SQL DATA** : La fonction peut lire des données variables dans la base de données, et donc, le résultat peut varier.

EXEMPLE

Exemple de fonction avec deux paramètres

```
DELIMITER |
DROP FUNCTION IF EXISTS verifier_date;
CREATE FUNCTION verifier_date (jour DATETIME, heure TIME)
RETURNS VARCHAR(10) DETERMINISTIC
BEGIN
    IF (
        NOT(MONTH(jour) IN (7,8) OR
        (MONTH(jour) = 6 AND DAY(jour) >=15) OR
        (MONTH(jour) = 9 AND DAY(jour)<=15)) AND
        ((DAYOFWEEK(jour) BETWEEN 2 AND 5)
        OR (DAYOFWEEK(jour) = 6 AND heure < '19:00:00')
        OR (DAYOFWEEK(jour) = 1 AND heure > '10:00:00')))
    THEN RETURN 'Incorrect';
    ELSE
    RETURN 'Correct';
    END IF;
END |
DELIMITER ;
```

EXEMPLE

Fonction avec 1 paramètre

```
DELIMITER |
DROP FUNCTION IF EXISTS controle_rencontre;
CREATE FUNCTION controle_rencontre (NomRencontre CHAR(20))
RETURNS VARCHAR(8) DETERMINISTIC
BEGIN
    IF (SELECT count(*) FROM passage P
        JOIN rencontre R
            ON R.REN_ID = P.REN_ID
        WHERE R.REN_NOM LIKE NomRencontre AND
        NOT(MONTH(P.PAS_DATE) IN (7,8) OR
        (MONTH(P.PAS_DATE) = 6 AND DAY(P.PAS_DATE) >=15) OR
        (MONTH(P.PAS_DATE) = 9 AND DAY(P.PAS_DATE)<=15)) AND
        ((DAYOFWEEK(P.PAS_DATE) BETWEEN 2 AND 5)
        OR (DAYOFWEEK(P.PAS_DATE) = 6 AND P.PAS_HEUREDEB < '19:00:00')
        OR (DAYOFWEEK(P.PAS_DATE) = 1 AND P.PAS_HEUREDEB > '10:00:00')) > 0
    THEN RETURN 'Pas Bien';
    ELSE
        RETURN 'Bien';
    END IF;
END |
DELIMITER ;
```



MYSQL - PROCÉDURES STOCKÉES

METTRE EN PLACE DES PROCÉDURES

PROCÉDURES STOCKÉES

Définition

Une procédure stockée, (stored procedure)

- concept utilisé en administration de base de données afin d'exécuter un ensemble d'instructions SQL.
- peut être appelée à tout moment par son nom afin d'exécuter celle-ci avec l'instruction **CALL**

Intérêts

- **Simplifier** : éviter la redondance de code.
- **Amélioration des performances** :
 - exécutée côté serveur : résultat envoyé directement au client
= réduit les échanges entre le client et le serveur,
- **Sécurité** : des applications peuvent avoir accès uniquement aux procédures stockées, sans avoir accès aux données des tables directement, et/ou s'assurer que l'accès aux données soit toujours effectué de la même manière.

SYNTAXE

Comme pour les fonctions, nous avons une syntaxe identique pour la création et la destruction.



Délimiteur : par défaut ; ce qui peut poser problèmes. Donc on peut changer de délimiteur ainsi le caractère qui délimite les instructions par l'instruction **DELIMITER**.

- *Cela n'agit que sur la session courante.*

```
CREATE PROCEDURE nom_procedure ([parametre1 [, parametre2, ...]])
BEGIN
    instruction1;
    instruction2;
    ...
    instruction3;
END;

DROP PROCEDURE nom_procedure;

CALL nom_procedure(...);

-- avec changement de délimiteur
DELIMITER | -- On change le délimiteur
CREATE PROCEDURE afficher_races()
| -- toujours pas de paramètres, toujours des parenthèses
BEGIN
|     SELECT id, nom, espece_id, prix
|     FROM Race; -- Cette fois, le ; ne nous embêtera pas
END| -- Et on termine bien sûr la commande CREATE PROCEDURE par
    notre nouveau délimiteur
DELIMITER ;
```

LA GESTION DES PARAMÈTRES

Gestion des paramètres

Un paramètre peut être de trois sens différents : entrant (**IN**), sortant (**OUT**), ou les deux (**INOUT**).

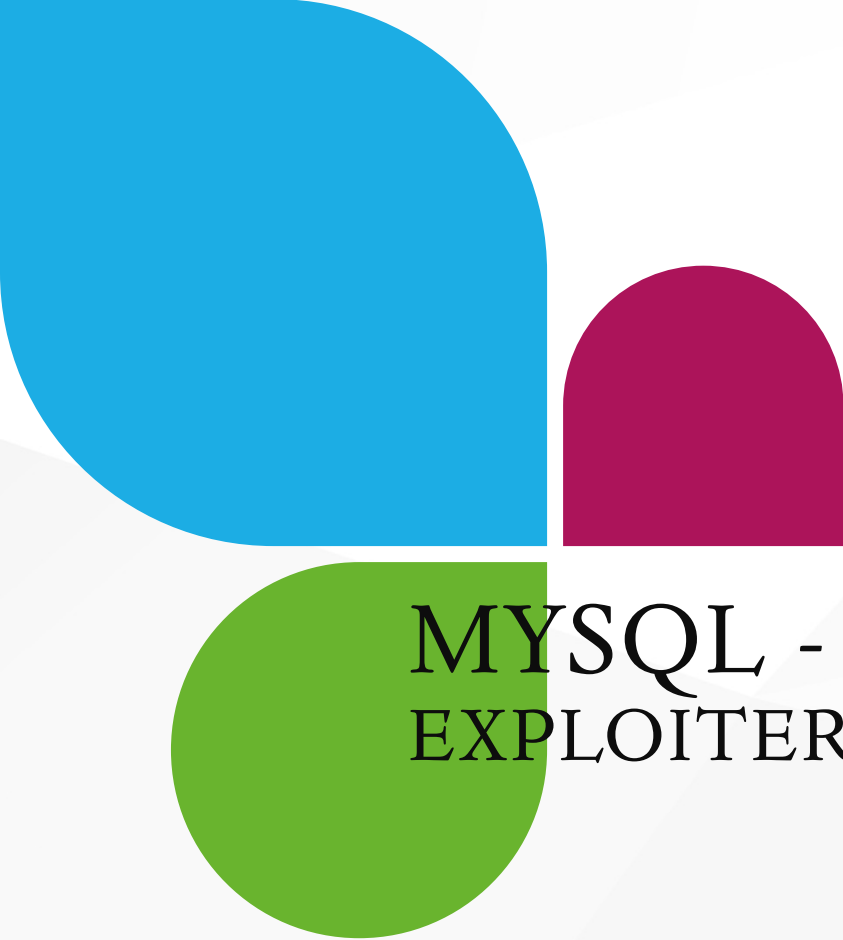
IN : c'est un paramètre "entrant". C'est-à-dire qu'il s'agit d'un paramètre dont la valeur est fournie à la procédure stockée. Cette valeur sera utilisée pendant la procédure (pour un calcul ou une sélection, par exemple).

OUT : il s'agit d'un paramètre "sortant", dont la valeur sera établie au cours de la procédure et qui pourra ensuite être utilisé en dehors de cette procédure.

INOUT : un tel paramètre sera utilisé pendant la procédure, verra éventuellement sa valeur modifiée par celle-ci, et sera ensuite utilisable en dehors.

Exemple

```
DELIMITER | -- Facultatif si votre délimiteur est toujours |
CREATE PROCEDURE afficher_race_selon_espece (IN p_espece_id INT)
| -- Définition du paramètre p_espece_id
BEGIN
| SELECT id, nom, espece_id, prix
| FROM Race
| WHERE espece_id = p_espece_id; -- Utilisation du paramètre
END |
DELIMITER ; -- On remet le délimiteur par défaut
```



MYSQL - CURSEURS

EXPLOITER UN SELECT

CURSEURS

Les curseurs permettent de parcourir un jeu de résultats d'une requête SELECT, quel que soit le nombre de lignes récupérées, et d'en exploiter les valeurs.

Quatre étapes sont nécessaires pour utiliser un curseur.

- Déclaration du curseur : avec une instruction DECLARE.
- Ouverture du curseur : on exécute la requête SELECT du curseur et on stocke le résultat dans celui-ci.
- Parcours du curseur : on parcourt une à une les lignes.
- Fermeture du curseur.

```
DELIMITER |
CREATE PROCEDURE nom_procedure()
BEGIN
    DECLARE curseur_client CURSOR FOR SELECT * FROM Client;

    -- ouverture du curseur
    OPEN curseur_client;

    -- parcours du curseur
    FETCH curseur_client INTO variables;
    /* REMARQUE
    variables correspond à autant de variables
    que l'on récupère de colonnes avec SELECT
    */
    -- fermeture du curseur
    CLOSE curseur_client;
END |
DELIMITER ;
```

PARCOURIR TOUS LES RÉSULTATS

Pour parcourir tous les résultats d'un curseur, on va pouvoir utiliser les structures de boucles telles que WHILE, REPEAT ou LOOP.

Mais il faut donc gérer la fin du parcours du curseurs soit quand il n'y a plus de données : ***ERROR 1329 (02000): No data - zero rows fetched, selected, or processed***

Pour cela, on va conditionner pour arrêter la boucle en déclarant un gestionnaire pour la condition NOT FOUND

```
DELIMITER |
CREATE PROCEDURE test_condition2(IN p_ville VARCHAR(100))
BEGIN
    DECLARE v_nom, v_prenom VARCHAR(100);
    DECLARE fin TINYINT DEFAULT 0; -- Variable locale utilisée pour stopper la boucle
    DECLARE curs_clients CURSOR
        FOR SELECT nom, prenom
        FROM Client
        WHERE ville = p_ville;

    -- Gestionnaire d'erreur pour la condition NOT FOUND
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;

    OPEN curs_clients;
    loop_curseur: LOOP
        FETCH curs_clients INTO v_nom, v_prenom;
        -- Structure IF pour quitter la boucle à la fin des résultats
        IF fin = 1 THEN
            LEAVE loop_curseur;
        END IF;

        SELECT CONCAT(v_prenom, ' ', v_nom) AS 'Client';
    END LOOP;
    CLOSE curs_clients;
END|
DELIMITER ;
```